

СЕМИНАР 3 · ВВЕДЕНИЕ В БАЗЫ ДАННЫХ

Первые шаги в SQL

Одна таблица на всё

order_no	order_date	customer_name	customer_phone	product_name	product_price	quantity
1001	03.03	Анна Петрова	+7 916 111-22-33	Кофемолка, Фильтры бумажные	4900, 350	1, 2
1002	04.03	Павел Ершов	+7 917 222-33-44	Кофемолка	4900	1
1003	05.03	Анна Петрова	+7 916 111-22-33	Термос	3200	1
1004	06.03	Мария Лебедева	+7 925 333-44-55	Турка медная	2600	1

ВОПРОС

Что здесь неудобно?

Три операции ломают эту таблицу

order_no	order_date	customer_name	customer_phone	product_name	product_price	quantity
1001	03.03	Анна Петрова	+7 916 111-22-33	Кофемолка, Фильтры бумажные	4900, 350	1, 2
1002	04.03	Павел Ершов	+7 917 222-33-44	Кофемолка	4900	1
1003	05.03	Анна Петрова	+7 916 111-22-33	Термос	3200	1
1004	06.03	Мария Лебедева	+7 925 333-44-55	Турка медная	2600	1

Жёлтое — один и тот же факт повторяется, красное — факт живёт только в этой строке

ОБНОВЛЕНИЕ	Сменился телефон — править строки во всех её заказах
ВСТАВКА	Новый товар не вставить: строке нужны заказ и клиент
УДАЛЕНИЕ	Удалили последнюю строку с туркой — товар исчез

1НФ: в ячейке — одно значение

БЫЛО

order_no	customer_name	product_name	product_price	quantity
1001	Анна Петрова	Кофемолка, Фильтры бумажные	4900, 350	1, 2

СТАЛО

order_no	customer_name	product_name	product_price	quantity
1001	Анна Петрова	Кофемолка	4900	1
1001	Анна Петрова	Фильтры бумажные	350	2
1002	Павел Ершов	Кофемолка	4900	1

Список стал строками, но таблица по-прежнему одна — и все три аномалии на месте.

2НФ: факт про всю строку, а не про часть ключа

БЫЛО

order_no	order_date	customer_name	product_name	product_price	quantity
1001	03.03	Анна Петрова	Кофемолка	4900	1
1001	03.03	Анна Петрова	Фильтры бумажные	350	2

ключ (order_no, product_name): order_no -> customer_name | product_name -> product_price

СТАЛО

orders

order_no	order_date	customer_name
1001	03.03	Анна Петрова

product

product_name	current_price
Кофемолка	4900
Фильтры бумажные	350

order_item

order_no	product_name	quantity
1001	Кофемолка	1
1001	Фильтры бумажные	2

2НФ ломается только при составном ключе: поле висит на половине ключа.

3НФ: факт про ключ, а не про соседнее поле

БЫЛО

order_no	order_date	customer_name	customer_phone
1001	03.03	Анна Петрова	+7 916 111-22-33
1003	05.03	Анна Петрова	+7 916 111-22-33

ключ order_no: order_no -> customer_name -> customer_phone

СТАЛО

orders

order_no	order_date	customer_id
1001	03.03	1
1003	05.03	1

customer

customer_id	name	phone
1	Анна Петрова	+7 916 111-22-33

3НФ ломается при любом ключе: поле висит на другом поле, а не на ключе.

Язык для работы с данными

SQL (Structured Query Language) — язык для работы с реляционными базами данных

Часть	Расшифровка	Для чего	Команды
DDL	Data Definition Language	Создание и изменение структуры	CREATE, ALTER, DROP
DML	Data Manipulation Language	Управление данными	INSERT, UPDATE, DELETE
DQL	Data Query Language	Запросы — отдельно выделяемая часть DML	SELECT
DCL	Data Control Language	Управление доступом к данным	GRANT, REVOKE
TCL	Transaction Control Language	Управление транзакциями и совместной обработкой данных	BEGIN, COMMIT, ROLLBACK

Выражения в SELECT

Пример	Результат	Что делает · когда использовать
<code>SELECT 2 + 2;</code>	4	Вычислить выражение без таблицы — проверить формулу, посчитать
<code>SELECT 2 + 2 AS four;</code>	four: 4	AS — имя столбца результата. Пишем всегда, иначе ?column?
<code>SELECT 1 AS a, 2 AS b;</code>	a: 1, b: 2	Несколько выражений через запятую — несколько столбцов
<code>WITH p AS (SELECT 5 AS x) SELECT x * 2 FROM p;</code>	10	«Переменная»: задать значение один раз и использовать по имени
<code>CREATE ...; INSERT ...;</code>		«;» завершает команду — несколько команд подряд в одной ячейке

Числовые типы

Имя	Размер	Описание	Диапазон
<code>smallint</code>	2 байта	Целое в небольшом диапазоне	−32768... +32767
<code>integer</code>	4 байта	Типичный выбор для целых чисел	−2147483648... +2147483647
<code>bigint</code>	8 байтов	Целое в большом диапазоне	−9223372036854775808... 9223372036854775807
<code>numeric, decimal</code>	Переменны й	Вещественное число с указанной точностью	До 131 072 цифр до десятичной точки и до 16 383 — после
<code>float4, real</code>	4 байта	Вещественное число с переменной точностью	Точность в пределах 6 десятичных цифр
<code>float8, double precision</code>	8 байтов	Вещественное число с переменной точностью	Точность в пределах 15 десятичных цифр

Столбцы с автоматическим увеличением

Имя	Размер	Описание	Диапазон
<code>smallserial</code>	2 байта	Небольшое целое с автоувеличением	1... 32767
<code>serial</code>	4 байта	Целое с автоувеличением	1... 2147483647
<code>bigserial</code>	8 байтов	Большое целое с автоувеличением	1... 9223372036854775807

- Не настоящие типы, а удобное средство для столбцов с уникальными числами
- `serial = serial4` → столбец `integer`; `bigserial = serial8` → столбец `bigint`

GENERATED ... AS IDENTITY

```
CREATE TABLE public.student (  
    student_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    name          text      NOT NULL  
);  
INSERT INTO public.student (name) VALUES ('Ada'), ('Alan');  -- id = 1, 2
```

Вариант	Поведение	Когда использовать
GENERATED ALWAYS	Номера выдаёт только СУБД; своё значение — ошибка	Обычный выбор для суррогатного ключа
GENERATED BY DEFAULT	Номер выдаёт СУБД, если значение не задано; можно задать своё	Перенос данных, где id уже есть
identity + PRIMARY KEY	Identity даёт NOT NULL, но не уникальность	Всегда: ключ защищает от повторов
serial	Старый синтаксис PostgreSQL: последовательность + DEFAULT	Узнавать в чужих схемах; в новых — identity

Целочисленные типы

- smallint, integer, bigint — числа без дробной части, различаются диапазоном
- Значение вне диапазона — ошибка
- integer — обычный выбор; smallint — экономия места; bigint — когда не хватает integer
- Стандарт SQL: integer (int), smallint, bigint. int2, int4, int8 — имена PostgreSQL

```
SELECT 40000::smallint;  
ERROR:  smallint out of range
```

Числа с фиксированной точностью

- `numeric` — точные вычисления, рекомендуется для денег; медленнее целых и `float`
- `numeric(точность, масштаб)`: `точность` — всего значимых цифр, `масштаб` — из них после точки
- `numeric` и `decimal` равнозначны

Число	Точность	Масштаб	Подходящий тип
23.5141	6	4	<code>numeric(6, 4)</code>
1234567890.12	12	2	<code>numeric(12, 2)</code>
42	2	0	<code>numeric(2, 0)</code> — целое

Типы с плавающей точкой

	numeric	real, double precision
Значения	Точные	Приближённые, с переменной точностью
Скорость	Медленнее	Быстрее
Когда брать	Деньги и всё, где важна точность	Измерения, дробные результаты вычислений

- float без точности — это double precision
- float(p): p — минимальная точность в двоичных цифрах

Приведение типов

Пример	Результат	Что делает · когда использовать
<code>'2016-10-13'::date</code>	2016-10-13	::тип — превратить значение в нужный тип (запись PostgreSQL)
<code>CAST('97' AS integer)</code>	97	То же по стандарту SQL — работает в любой СУБД
<code>date '2016-10-13'</code>	2016-10-13	Константа сразу нужного типа — для дат, времени, интервалов
<code>10 = '10'</code>	true	Неявно: строка в кавычках подстраивается под соседа
<code>10.56::int</code>	11	Дробное → целое округляет, а не отбрасывает дробную часть
<code>'abc'::int</code>	ERROR	Значение не читается как тип — данные не того формата

Арифметика и округление

Пример	Результат	Что делает · когда использовать
<code>2 + 3 * 4</code>	14	+ - * — порядок как в математике
<code>7 / 2</code> <code>7 / 2.0</code>	3 3.5	Целое / целое = целое; нужна дробь — пишем 2.0
<code>93 % 13</code>	2	Остаток: распределение по кругу, чётность
<code>2 ^ 10</code>	1024	Степень
<code>ceil(7 / 2.0)</code>	4	Вверх: сколько нужно партий, считая неполную
<code>floor(3.9)</code>	3	Вниз: сколько полных партий
<code>round(2.567, 2)</code>	2.57	К ближайшему; n — знаков после точки

Строковые типы

Имя	Описание
<code>character varying(n)</code> , <code>varchar(n)</code>	Строка ограниченной переменной длины
<code>character(n)</code> , <code>char(n)</code> , <code>bpchar(n)</code>	Строка фиксированной длины, дополненная пробелами
<code>text</code>	Строка неограниченной переменной длины

Как хранится значение 'ABC'

Тип	Хранится	Почему
<code>char(5)</code>	'ABC '	Дополняется пробелами до n символов
<code>varchar(5)</code>	'ABC'	Сохраняется как есть; длиннее 5 — ошибка
<code>text</code>	'ABC'	Ограничения длины нет

Строковые типы: длина и псевдонимы

- `text` — встроенный тип PostgreSQL, строковые функции работают с `text`
- `varchar` — псевдоним `character varying`; `char` и `bpchar` — псевдонимы `character`
- `n` — от 1 до 10485760 символов
- Без длины: `varchar` и `bpchar` — любая длина, `char` — это `char(1)`
- Короткие строки занимают размер + 1 байт, длинные — + 4 байта и сжимаются; максимум ~1 ГБ
- Нет предела длины — `text` или `varchar` без `n`, а не большое `n`

Работа со строками

Пример	Результат	Что делает · когда использовать
'ABC '	ABC	Одинарные кавычки — строковое значение
"ABC"	ERROR	Двойные кавычки — имя столбца или таблицы, не строка
'ABC ' '-' 'DEF'	ABC-DEF	склеивает строки — собрать текст из частей
'Мест: ' 97	Мест: 97	Число или дата при склейке сами станут текстом
'ABC' = 'abc' '10' < '9'	false true	Сравнение посимвольное и с учётом регистра
lower('ABC') = 'abc'	true	Привести к одному регистру — сравнить без учёта регистра

Типы даты/времени

Имя	Размер	Описание	Наименьшее значение	Наибольшее значение	Точность
timestamp [(p)] [without time zone]	8 байтов	Дата и время (без часового пояса)	4713 до н. э.	294 276 н. э.	1 микросекунда
timestamp [(p)] with time zone	8 байтов	Дата и время (с часовым поясом)	4713 до н. э.	294 276 н. э.	1 микросекунда
date	4 байта	Дата (без времени суток)	4713 до н. э.	5 874 897 н. э.	1 день
time [(p)] [without time zone]	8 байтов	Время суток (без даты)	00:00:00	24:00:00	1 микросекунда
time [(p)] with time zone	12 байтов	Время дня (без даты) с часовым поясом	00:00:00+1559	24:00:00–1559	1 микросекунда
interval [поля] [(p)]	16 байтов	Временной интервал	–178 000 000 лет	178 000 000 лет	1 микросекунда

Подробнее о типах даты/времени

- timestamp = timestamp without time zone (стандарт SQL)
- timestamptz = timestamp with time zone (расширение PostgreSQL)
- (p) у time, timestamp, interval — сколько знаков после точки в секундах: от 0 до 6, по умолчанию не ограничено

Выражение	Результат
<code>now()</code>	2026-09-29 14:14:03.214415+03
<code>now()::timestamptz(0)</code>	2026-09-29 14:14:03+03
<code>now()::timestamp(0)</code>	2026-09-29 14:14:03
<code>'1999-01-08 04:05:06 -8:00'::timestamptz</code>	1999-01-08 15:05:06+03

Арифметика дат и времени

Пример	Результат	Что делает · когда использовать
<code>date '2016-01-31' + 30</code>	<code>2016-03-01</code>	+ число — плюс дни, результат date
<code>date '2016-01-31' + interval '1 month'</code>	<code>2016-02-29 00:00:00</code>	+ interval — календарно, результат timestamp
<code>(...)::date</code>	<code>2016-02-29</code>	Отбросить время, оставить дату
<code>3 * interval '40 minutes'</code>	<code>02:00:00</code>	Число × интервал = длительность
<code>time '17:00' - time '14:05'</code>	<code>02:55:00</code>	Момент – момент = interval
<code>date '2016-03-01' - date '2016-01-31'</code>	<code>30</code>	Дата – дата = число дней

Логический тип

Имя	Размер	Описание
-----	--------	----------

<code>boolean</code>	1 байт	Состояние: истина или ложь. Значения: <code>true</code> , <code>false</code>
----------------------	--------	--

Соответствуют «true»	Соответствуют «false»
----------------------	-----------------------

<code>true</code> <code>yes</code> <code>on</code> <code>1</code> <code>t</code>	<code>false</code> <code>no</code> <code>off</code> <code>0</code> <code>f</code>
--	---

- Третье состояние — «unknown», представляется значением `NULL`
- В запросах предпочтительно писать `TRUE` и `FALSE` — как в стандарте SQL

Сравнения и логика

Пример	Результат	Что делает · когда использовать
10 <> 10.5	true	= равно, <> и != не равно
10 > 100	false	< > <= >= — числа, даты, строки
5 BETWEEN 1 AND 10	true	В диапазоне, границы включены — удобно для CHECK
true AND false	false	AND — истинны оба условия
true OR false	true	OR — истинно хотя бы одно
NOT true	false	NOT — отрицание

NULL в выражениях

Пример	Результат	Что делает · когда использовать
1 + NULL 'ABC' NULL	NULL NULL	Любая операция с неизвестным даёт неизвестное
NULL = NULL	NULL	Равны ли два неизвестных — неизвестно. Через = на NULL не проверяют
true OR NULL false AND NULL	true false	Ответ не зависит от неизвестного — он известен
true AND NULL	NULL	Ответ зависит от неизвестного — неизвестен
x IS NULL	true / false	Единственная правильная проверка «значения нет»
NULL IS NOT DISTINCT FROM NULL	true	Сравнение, где NULL равен NULL

Полезные ссылки и используемые источники

<https://postgrespro.ru/docs/postgresql/17/datatype>

Выбираем тип, который передаёт смысл

Что храним	Тип	Почему
Деньги, оценки	<code>numeric(p, s)</code>	Точные десятичные
Измерения, приближённые расчёты	<code>real, double precision</code>	Плавающая точка, быстро
Текст без правила о длине	<code>text</code>	Основной вариант для строк
Текст, где длина — правило	<code>varchar(n)</code>	Проверяет длину
Код фиксированной длины	<code>char(n)</code>	Дополняется пробелами
Календарная дата	<code>date</code>	День рождения, срок
Момент времени, событие	<code>timestampz</code>	Однозначная точка на шкале
«Местное время» без пояса	<code>timestamp</code>	Локальное расписание
Локальный суррогатный ключ	<code>bigint identity / serial</code>	Номера выдаёт СУБД

Ограничения — исполняемая документация схемы

Ограничение	Что делает	Пример
NOT NULL	Отсутствующее значение запрещено	<code>name text NOT NULL</code>
CHECK	Логическое правило для строки	<code>CHECK (credits BETWEEN 1 AND 12)</code>
UNIQUE	Запрещает повторы; NULL допускается несколько раз	<code>email text UNIQUE</code>
PRIMARY KEY	UNIQUE + NOT NULL, главный идентификатор строки	<code>student_id integer PRIMARY KEY</code>
REFERENCES	Внешний ключ: значение должно быть в другой таблице	<code>REFERENCES public.student (student_id)</code>

CREATE TABLE

```
CREATE TABLE public.student (  
    student_id integer    PRIMARY KEY,  
    name            text    NOT NULL,  
    email           text    UNIQUE,  
    semester        smallint CHECK (semester BETWEEN 1 AND 14)  
);  
  
CREATE TABLE public.course (  
    course_id integer    PRIMARY KEY,  
    title           text    NOT NULL,  
    credits         smallint NOT NULL CHECK (credits BETWEEN 1 AND 12)  
);
```

Столбец: имя, тип, ограничения. Свои таблицы создаём в схеме public и пишем имя целиком: public.имя

Составной ключ и внешние ключи

```
CREATE TABLE public.enrollment (  
    student_id integer REFERENCES public.student (student_id),  
    course_id integer REFERENCES public.course (course_id),  
    enrolled_at timestamptz NOT NULL DEFAULT now(),  
    PRIMARY KEY (student_id, course_id)  
);
```

- REFERENCES — запись можно добавить только для существующих студента и курса
- PRIMARY KEY (student_id, course_id) — составной ключ, отдельной строкой: студент записан на курс один раз
- DEFAULT now() — значение, если при вставке столбец не указан

INSERT и сработавшие ограничения

```
INSERT INTO public.student (student_id, name, semester)
VALUES (1, 'Ada Lovelace', 1),
       (2, 'Alan Turing', 2);
```

Попытка вставить	Что ответит PostgreSQL
студента с semester = 20	violates check constraint
студента без name	null value violates not-null constraint
ещё раз student_id = 1	duplicate key value violates unique constraint
запись на курс для student_id = 7	violates foreign key constraint
ту же запись (1, 10) второй раз	duplicate key value — составной ключ

Служебные команды

Пример	Результат	Что делает · когда использовать
<code>public.student</code>		Полное имя: схема + таблица. Для своих таблиц пишем всегда
<code>SELECT * FROM public.student;</code>	все строки	Посмотреть таблицу целиком — проверить, что вставилось
<code>DROP TABLE public.student;</code>	DROP TABLE	Удалить таблицу вместе с данными, без корзины
<code>DROP TABLE IF EXISTS public.student;</code>	DROP TABLE	Без ошибки, если таблицы нет — перед CREATE, чтобы перезапустить
<code>DROP ... enrollment, потом student</code>		Сначала таблицу со ссылкой (FK), потом основную
<code>CREATE TABLE IF NOT EXISTS ...</code>		Создать, только если таблицы ещё нет